

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

INFORME DE USO CRÍTICO Y AUDITORÍA DE ALGORITMOS

Profesor(a):	Ariel Adara Mercado Martínez
Asignatura:	Estructuras de Datos y Algoritmos I
Grupo:	5
No. de Práctica(s):	Proyecto Final
Integrante(s):	González Mateo Carlos Enriquez Bahena Fernando Israel De Jesús Carranza Giovanni Gabriel
Nombre de equipo:	Tosti
Semestre:	2026-2
Fecha de entrega:	11 de junio de 2026

CALIFICACIÓN: _____

ENTRADA 1: DISEÑO DE LA COLA CIRCULAR (SCHEDULER CPU)

- **Módulo Objetivo:** Estructura de control para la planificación Round Robin.
- **Técnica de Prompting:** *Roleplay con Restricción de Optimización de Memoria.*
- **Prompt Enviado:** > "Actúa como un experto en sistemas operativos y estructuras de datos en C. Necesito diseñar una cola circular para un planificador Round Robin. Cada nodo debe contener una estructura llamada *Process* con los campos: *pid*, *burst_time*, *remaining_time*, *priority*, *memory_required* y un enum para los estados. Optimiza la cola utilizando un puntero únicamente al nodo *tail* (final) para asegurar que las operaciones de *enqueue* y *dequeue* sean de complejidad constante $O(1)$."
- **Respuesta Inicial de la IA:** Proveía una estructura estándar y funciones básicas de inserción y extracción. Sin embargo, asumía que al desencolar siempre quedarían nodos en la lista.
- **Análisis Crítico y Errores Detectados:** Al auditar la función de extracción *dequeue()*, se detectó un fallo crítico de lógica: la IA redirigía el puntero del frente (*q->tail->next = front->next*) sin validar si el nodo extraído era el último elemento restante en la cola. Esto provocaba que *q->tail* apuntara a una dirección de memoria ya liberada (*Dangling Pointer*), generando un fallo de segmentación (*Segmentation Fault*) en la siguiente iteración.
- **Corrección y Validación Realizada:** Modifiqué manualmente el algoritmo introduciendo una cláusula de control estricta para el caso base de un solo nodo restante:
- C

```
if (q->tail == front) {  
    q->tail = NULL;  
} else {  
    q->tail->next = front->next;  
}
```

-
- Se validó de forma estricta en consola vaciando colas de prueba de tamaños variable (1 a 50 procesos).

ENTRADA 2: ADMINISTRADOR DE MEMORIA (LISTA DOBLE Y ALGORITMOS GREEDY)

- **Módulo Objetivo:** Asignación dinámica de bloques de RAM mediante *First Fit*, *Best Fit* y *Worst Fit*.
- **Técnica de Prompting:** *Diseño Algorítmico Basado en Restricciones de Comportamiento.*
- **Prompt Enviado:**
"Ayúdame a diseñar un administrador de memoria en C usando una lista doblemente ligada basada en bloques (*MemoryBlock*). Cada bloque debe registrar su inicio, tamaño, si está libre y el PID asignado. Implementa las funciones de asignación mediante las estrategias Greedy: *First Fit*, *Best Fit* y *Worst Fit*. Si el bloque elegido

es más grande que lo solicitado, debe fragmentarse en un nuevo nodo. Además, implementa una función de coalescencia que fusione bloques libres adyacentes al liberar un proceso."

- **Respuesta Inicial de la IA:** Entregó la lógica para buscar los bloques y una rutina recursiva de coalescencia.
- **Análisis Crítico y Errores Detectados:** 1. En la función de fragmentación de bloques (`splitBlock`), la IA modificó correctamente los punteros `next` del bloque base y del bloque sobrante (`leftover`), pero **olvidó por completo actualizar el puntero `prev`** del nodo que originalmente se encontraba después del bloque modificado. Esto rompía la propiedad bidireccional de la lista doblemente ligada. 2. El algoritmo de coalescencia propuesto por la IA realizaba una recursión innecesaria que incrementaba el uso de la pila de llamadas, arriesgando un desbordamiento (*Stack Overflow*) con mapas de memoria altamente fragmentados.
- **Corrección y Validación Realizada:** 1. Corregí los enlaces bidireccionales en la fragmentación inyectando la validación:
- C

```
if (block->next != NULL) {
    block->next->prev = leftover;
}
```

-
- 2. Reemplacé la función recursiva de coalescencia por un enfoque iterativo lineal adaptativo con la instrucción `continue`, asegurando que la fusión ocurra sobre memoria contigua en una sola pasada con costo espacial auxiliar $O(1)$.

ENTRADA 3: ANÁLISIS DE COMPLEJIDAD ASINTÓTICA

- **Módulo Objetivo:** Sección Teórica y Justificación Matemática del Reporte.
- **Técnica de Prompting:** *Sustentación de Teoría Algorítmica.*
- **Prompt Enviado:**
"Dame el análisis de complejidad temporal y espacial en el peor y mejor caso para los algoritmos Best Fit y Worst Fit implementados sobre la lista doblemente ligada del proyecto."
- **Respuesta Inicial de la IA:** La IA afirmó erróneamente que, al tratarse de algoritmos Greedy (Ávidos), la complejidad temporal para encontrar el bloque ideal era de tiempo constante $O(1)$ en el mejor de los casos debido a la naturaleza inmediata del paradigma.
- **Análisis Crítico y Errores Detectados:** Se detectó una falla teórica severa en la respuesta de la IA. Aunque la toma de decisiones del paradigma Greedy es directa, la localización del bloque óptimo (*Best Fit*: el que deje menos espacio sobrante; *Worst Fit*: el que deje más espacio) está restringida por la estructura de datos utilizada. Al ser una lista doblemente ligada no ordenada, el algoritmo está estrictamente obligado a examinar todos los m bloques de la lista para asegurar cuál de ellos cumple con la condición extrema. No existe un atajo matemático posible en una lista lineal desordenada.
- **Corrección y Validación Realizada:** Rechacé el análisis inicial de la IA y reestructuré la sección matemática del reporte técnico, demostrando formalmente

mediante sumatorias que tanto el mejor como el peor escenario técnico están acotados por una complejidad temporal estricta de $\Theta(m)$.

ENTRADA 4: INTEGRACIÓN INTERLENGUAJE Y MANEJO DE ENTRADAS CSV

- **Módulo Objetivo:** Parseo de cadenas de texto y exportación de métricas de rendimiento para Python.
- **Técnica de Prompting:** *Chain-of-Thought (Cadena de Pensamiento)*.
- **Prompt Enviado:**
"Muestra el procedimiento detallado en C para abrir un archivo 'procesos.csv', brincar la fila del encabezado de texto de manera segura, parsear cada línea con variables enteras e introducirlas a la cola circular. Al final, escribe un archivo 'resultados.csv' ordenado secuencialmente por PID."
- **Respuesta Inicial de la IA:** Proponía leer el archivo con `fscanf` directo usando comas como delimitadores.
- **Análisis Crítico y Errores Detectados:** El uso de `fscanf` directo con formatos del tipo `%d,%d` es altamente inestable si el archivo CSV contiene espacios en blanco al final de la línea o saltos de carro inesperados (`\r\n` de entornos Windows). Esto provocaba que el puntero del archivo entrara en un ciclo infinito de lectura fallida o corrompiera los identificadores de los procesos (PID).
- **Corrección y Validación Realizada:** Diseñé un flujo de lectura de doble capa mucho más robusto: primero se extrae la línea completa del archivo CSV de forma segura como una cadena de texto controlada usando `fgets(buffer, sizeof(buffer), csv_in)`, y posteriormente se realiza el parseo de datos en memoria utilizando la función estructurada `sscanf`. Esto garantizó la inmunidad del simulador maestro ante caracteres ocultos de control de formato entre sistemas operativos (Ubuntu/Windows).

